

# zkASM: Zero-Knowledge Assembly Language

## A Deep Dive into the go-corset Implementation

go-corset Project

Consensys

January 28, 2026



# What is zkASM?

## Zero-Knowledge Assembly Language

- Low-level language for ZK circuits
- Part of go-corset compiler
- Explicit register control
- Field-agnostic design

## Key Features

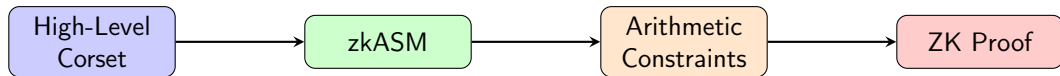
- Strong type system (u1 to u256)
- Multi-value returns
- Function composition via buses
- Automatic constraint generation

```
fn add(a u32, b u32)
  -> (sum u32, carry u1) {
    carry, sum = a + b
    return
  }
```

## Generated Constraint:

$$2^{32} \cdot \text{carry} + \text{sum} = a + b$$

# Why zkASM?



## Advantages

- Direct control over constraints
- Predictable constraint counts
- Optimized for specific algorithms
- Reusable function libraries

## Use Cases

- EVM opcode implementations
- Cryptographic primitives (Blake2f)
- Arithmetic circuits (Division, Modexp)
- Bit manipulation routines

## Unsigned Integer Types

| Type | Range              |
|------|--------------------|
| u1   | $[0, 1]$           |
| u8   | $[0, 255]$         |
| u16  | $[0, 65535]$       |
| u32  | $[0, 2^{32} - 1]$  |
| u128 | $[0, 2^{128} - 1]$ |
| u256 | $[0, 2^{256} - 1]$ |

## Type Safety

- Compiler verifies bit-width balance
- Carry bits explicitly captured
- No implicit overflow

```
;; 8-bit + 8-bit = 9-bit result  
var carry u1  
var result u8  
carry, result = a + b    ;; OK!  
  
result = a + b    ;; ERROR: missing carry
```

# Function Declarations

```
fn divide(dividend u16, divisor=1 u16)
  -> (quotient u16, remainder u16) {

  var sum u32
  var witness u16
  var witsum u17

  quotient, remainder, witness, sum, witsum = dividend / divisor
  return
}
```

## Input Parameters

- Named with explicit types
- Optional default values (=1)
- Used for padding in traces

## Output Parameters

- Multiple return values
- Explicit output types
- Optional padding values

# Control Flow

```
fn counter(n u256, m u256) -> (r u256) {  
    var i u256  
    var c0, c1 u1  
  
    r = n  
    i = m  
  
loop:  
    if i == 0 goto exit  
    c0, r = r + 1  
    c1, i = i - 1  
    goto loop  
  
exit:  
    return  
}
```

## Control Flow Constructs

- Labels (loop:, exit:)
- Conditional jumps (if ... goto)
- Unconditional jumps (goto)
- Function return (return)

## Comparison Operators

- ==, != (equality)
- <, <= (less than)
- >, >= (greater than)

# Function Calls and Buses

```
;; Library function
fn square(x u16) -> (r u32) {
    var hi u16
    hi, r = x * x
    return
}

;; Caller function
fn sum_of_squares(a u16, b u16)
-> (r u32) {
    var sq_a, sq_b u32
    var carry u1

    sq_a = square(a)
    sq_b = square(b)
    carry, r = sq_a + sq_b
    return
}
```

## Bus Protocol

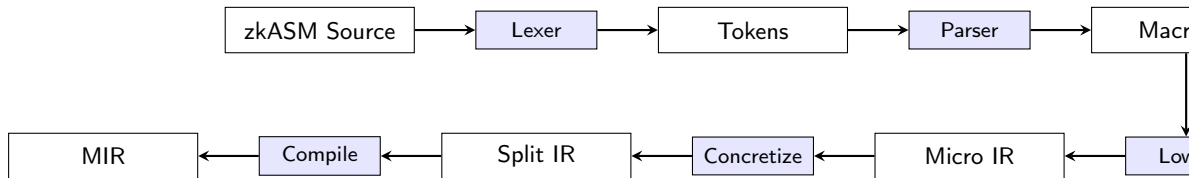
- Each function → unique bus ID
- Inputs = address lines
- Outputs = data lines
- Resolved at link time

## Constraints

- Caller writes address
- Callee provides data
- Lookup verifies match



# Pipeline Overview



## Front-end

- Lexer: Tokenization
- Parser: AST construction
- Linker: Bus resolution
- Validator: Type checking

## Back-end

- Lower: Macro  $\rightarrow$  Micro
- Concretize: Field adaptation
- Compile: Constraint generation

# Macro to Micro Lowering

## Macro Instructions

- High-level, user-facing
- Complex operations
- Control flow abstractions

Assign

Division

IfGoto

Call

Return

## Micro Instructions

- Primitive operations
- Direct constraint mapping
- Explicit control flow

Assign (polynomial)

Skip (conditional)

Jmp (unconditional)

InOut (bus I/O)

Ret, Fail

**Example:** if  $x == 5$  goto target  $\Rightarrow$

- 1 Skip{x, 5, skip=1} – skip next if  $x \neq 5$
- 2 Jmp{fallthrough} – go to next instruction
- 3 Jmp{target} – jump to target

# Vectorization Optimization

## Before Vectorization

```
;; 3 separate instructions  
a = x + 1  
b = y * 2  
c = z - 3
```

⇒ 3 constraint sets

## After Vectorization

```
;; 1 merged instruction  
{a, b, c} = {x+1, y*2, z-3}
```

⇒ 1 constraint set

## Merge Conditions

- No register write conflicts
- No branch targets between
- Sequential in original code

## Benefits

- Fewer constraints
- Smaller proof size
- Faster verification

# The Field Independence Problem

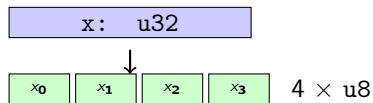
## Problem

zkASM programs use types up to u256, but different ZK systems use different finite fields:

| Field     | Max Bits |
|-----------|----------|
| BLS12-377 | 252      |
| KoalaBear | 30       |
| GF(8209)  | 12       |

A 256-bit register doesn't fit in *any* of these fields!

**Solution: Register Splitting**  
Split wide registers into limbs:



$$x = x_0 + 2^8 x_1 + 2^{16} x_2 + 2^{24} x_3$$

# Constraint Transformation

**Original Constraint** (field-independent):

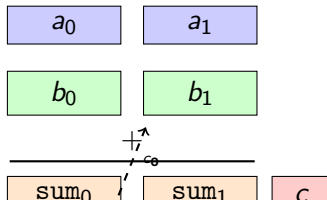
$$\text{carry} \cdot 2^{16} + \text{sum} = a + b$$

**After Splitting** for 8-bit field max:

$$c_0 \cdot 2^8 + \text{sum}_0 = a_0 + b_0 \quad (\text{lower byte})$$

$$\text{carry} \cdot 2^8 + \text{sum}_1 = a_1 + b_1 + c_0 \quad (\text{upper byte})$$

where  $c_0$  is an *internal* carry between limbs.



# Assignment Constraints

For assignment  $t_n, \dots, t_0 = \text{expr}$ :

$$\sum_{i=0}^n t_i \cdot 2^{w_i} = \text{eval}(\text{expr})$$

where  $w_i = \sum_{j=0}^{i-1} \text{width}(t_j)$ .

## Example 1: Simple

```
result = a + b
```

Constraint:  $\text{result} = a + b$

## Example 2: With Carry

```
carry, result = a + b
```

Constraint:  $2^n \cdot \text{carry} + \text{result} = a + b$

## Example 3: Multiplication

```
hi, lo = x * y    ;; x, y: u16, hi, lo: u16
```

Constraint:  $2^{16} \cdot \text{hi} + \text{lo} = x \cdot y$

# Division Constraints

```
q, r, w, sum, witsum = x / y
```

## Generated Constraints:

$$\begin{aligned} \text{sum} &= q \cdot y + r \\ \text{witsum} &= r + w + 1 \\ \text{sum} &= x \\ \text{witsum} &= y \end{aligned}$$

## Purpose:

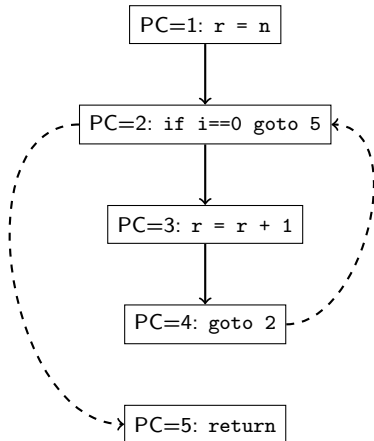
- Eq 1:  $q \cdot y + r = \text{sum}$
- Eq 3:  $\text{sum} = x$
- Together:  $x = q \cdot y + r$

## Witness proof ( $r < y$ ):

- $w = y - r - 1 \geq 0$
- Eq 2:  $r + w + 1 = \text{witsum}$
- Eq 4:  $\text{witsum} = y$
- Thus:  $w = y - r - 1$

# Program Counter Constraints

Multi-line functions require PC tracking:



## Constraints:

- Sequential:  $PC_{i+1} = PC_i + 1$
- Jump:  $PC_{i+1} = \text{target}$
- Return:  $RET_i = 1$

## Guard formula:

$$(PC_i = p) \Rightarrow C_p$$

Each instruction's constraints are guarded by its PC value.



# Trace Generation

## Executor State

```
type State struct {  
    pc      uint      // Program counter  
    terminal bool      // Done?  
    state   []big.Int // Register values  
    io      Map        // I/O for calls  
}
```

## Execution Loop

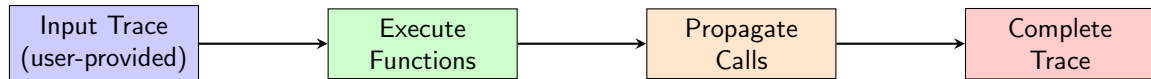
```
for !state.terminal {  
    insn := program[state.pc]  
    state.pc = insn.Execute(state)  
}
```

## Trace Structure

| Row | PC | a | b |
|-----|----|---|---|
| 0   | 1  | 5 | 3 |
| 1   | 2  | 5 | 3 |
| 2   | 3  | 6 | 2 |
| 3   | 2  | 6 | 2 |
| 4   | 3  | 7 | 1 |
| 5   | 2  | 7 | 1 |
| 6   | 4  | 8 | 0 |

Each row = one execution step

# Trace Propagation



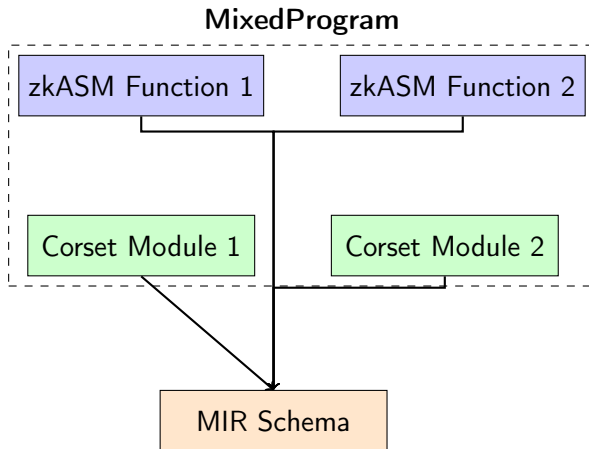
## Automatic Call Propagation

- User provides instances for top-level functions
- Executor runs functions, recording all internal calls
- Called functions automatically get their instances generated
- Final trace includes all function instances

## Instance Caching

- Same inputs  $\Rightarrow$  cached outputs
- Avoids redundant computation
- Thread-safe with mutex protection

# Mixed Program Model



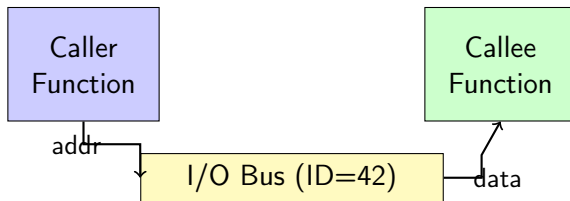
## zkASM Functions

- Compiled from `.zkasm` files

## Corset Modules

- Compiled from `.lisp` files

# Bus Communication



Address: input values (args)  
Data: output values (returns)

## Bus Protocol

- Each function assigned unique bus ID at link time
- Caller writes inputs to address lines
- Callee provides outputs on data lines

# Key Takeaways

## Language Features

- Strong type system (u1-u256)
- Explicit carry/overflow handling
- Clean function composition
- Field-agnostic programming

## Compilation

- Multi-stage pipeline
- Macro  $\rightarrow$  Micro lowering
- Vectorization optimization
- Automatic register splitting

## Constraint Generation

- Direct polynomial mapping
- PC-guarded multi-line functions
- Division with witness proofs
- Bus-based function calls

## Execution

- Deterministic trace generation
- Automatic call propagation
- Instance caching
- Thread-safe execution

**zkASM enables precise, portable, and efficient ZK circuit programming**

## Source Organization

pkg/asm/

- assembler/ – lexer, parser, linker, validator
- compiler/ – constraint generation
- io/ – core abstractions
  - macro/ – macro instructions
  - micro/ – micro instructions
- program/ – module wrappers

## Test Coverage

- 34+ unit test files
- Arithmetic operations
- Control flow patterns
- Function calls
- Bit manipulation
- Field-specific tests

## Benchmarks

- EVM opcodes
- Blake2f
- Division/Modexp
- Shift operations

# Questions?

`github.com/consensus/go-corset`